C% CCOVERT

SYSTEM REFERENCE · 2026

System Documentation

Internal design, application programming interfaces, services, and operational controls for the CCOverT coral-cover research workspace.

Annual live coral-cover projections

Research workspace for Puerto Princesa City, Palawan. Users can explore prediction history and reports; researchers manage the shared model configuration; developers operate account and service controls.

React · TypeScript | Express API | FastAPI model | MongoDB

Prepared from the application source and repository documentation. Deployment configuration values and credentials are intentionally excluded.

01 · INTERNAL DESIGN

System architecture

CCOverT separates the browser experience, authenticated application API, numerical model boundary, and persistent storage. The browser calls the Express API under `/api/v1`. The Python model service is an internal service and is not exposed to the browser.

React web client
Vite build · Nginx

→

Express API
Sessions · role checks

→

Model + database
FastAPI · MongoDB

Client workspace

Runs authorized predictions, reviews annual outputs and computation details, views personal history, and creates evidence-based reports.

Researcher workspace

Maintains one active shared model configuration. Parameter changes require a source or rationale and are used by later client predictions.

Developer workspace

Manages accounts, system announcement, email configuration, backups, and operational status. Secret values are not displayed.

Model and evidence

FastAPI owns numerical calculations. Prediction records retain their input, model version, source coverage, warnings, and year-by-year computation values.

Prediction flow

1. The verified user submits a profile, baseline, horizon, study area, and any selected source data.
2. The API validates access and inputs, resolves the active configuration, and calls the private model service.
3. The model returns the annual series and decomposition; the API stores the result and provenance for the account.
4. The browser renders the result and offers authorized history and report downloads.

> **Profiles:** Paper uses the active configuration and refuses a run when required values are missing. Scenario runs require explicit, reasoned assumptions. Demo uses synthetic values and is disabled in production.

02 · APPLICATION PROGRAMMING INTERFACES (API)

Browser API reference

Base path: `/api/v1`. Requests use JSON unless a route accepts a multipart file. Session-authenticated browser requests include the HttpOnly session cookie. Mutations also apply same-origin checks.

Area	Routes	Purpose / access
Authentication	`POST /auth/signup`, `/auth/verify`, `/auth/login`, `/auth/logout`, `/auth/forgot-password`, `/auth/reset-password`; `GET /auth/me`	Account creation and verification, session lifecycle, password recovery, current user.
Predictions	`POST /predictions`; `GET /predictions`, `/predictions/:id`, `/predictions/:id/download`, `/predictions/:id/reports`	Verified user runs; owned records and downloads in Markdown, CSV, or JSON. Supports `Idempotency-Key`.
Model	`GET /model/status`, `/conditions`, `/parameters`, `/versions`; `PUT /model/configuration`; version review/activation routes	Read model readiness and parameter values; researcher changes shared configuration; role checks apply to writes.
Data imports	`GET /data-imports`, `/schema/:kind`; `POST /data-imports`, `/:id/estimate-g`	Researcher-owned sourced SST, tourism, and coral-cover data; admin reviews as configured.
AI reports and uploads	`GET /ai/status`, `GET/POST /ai/reports`; `GET/POST/DELETE /uploads`	Reports from selected authorized evidence; private user uploads with bounded formats and size.
Settings and operations	`PATCH /settings`; `GET /developer/status`, `PATCH /developer/config`, `PATCH /developer/smtp`, account and backup routes	User preferences; admin-only operational configuration, account administration, and backups.

Response and failure behavior

Validation failures use HTTP 400; oversized uploads use 413; rate limits use 429. Errors include a stable code, message, optional details, and request ID. A prediction is not stored when the request is refused or model execution fails. A repeated idempotency key with a changed body returns a conflict.

> The internal model API is a separate service boundary. It requires the shared `x-service-token` header except health probes; it is not part of the public browser API.

03 · SERVICES

Runtime services

Service	Technology	Responsibility and boundary
Web	React, TypeScript, Vite, Nginx	Public landing, sign-in, and responsive client/researcher/developer screens. Container listens on port 80; Compose binds host port 8088 to loopback by default.
API	Node.js, Express	Authentication, authorization, validation, persistence, prediction orchestration, report generation, and operational routes. Container listens on port 4000.
Model	Python, FastAPI	Numerical calculation, model validation and calibration. Container listens on port 8000 on the private backend network.
Database	MongoDB 7	Users, configuration versions, predictions, source records, reports, and other application state. Compose attaches a persistent named volume.
Mail	Configured SMTP provider	Verification and password recovery messages. Production credentials stay in environment configuration.
Optional providers	AI and environmental integrations	Can provide evidence interpretation or environmental data. AI cannot change numerical prediction output; deterministic reports are used when no provider is configured.

Health and readiness

The API exposes `/health`, `/ready`, and admin-protected `/metrics`. The model exposes `/health` and `/ready`. Container health checks gate startup dependencies for the local Compose stack.

The Compose file places MongoDB and the model service on an internal backend network. The API connects to both networks; only the web container publishes a loopback-bound host port by default. Production proxy and domain routing are deployment-specific.

04 · CONTROLS

Access, data, and operational controls

Authentication

Session cookie is HttpOnly. Email verification gates prediction and evidence features for client accounts. Password reset tokens are single use and responses do not reveal whether an email exists.

Role authorization

Client, researcher, and admin routes apply server-side role checks. The server derives the account from the session; it does not trust a client-supplied user ID.

Record ownership

Prediction, upload, dataset, and report reads and updates are scoped to the authenticated user or authorized role. Upload bytes are private and are not served by the web app.

Request safeguards

Schema validation, same-origin checks for mutations, rate limits, bounded JSON and file sizes, password controls, and idempotent prediction submissions reduce accidental or unauthorized changes.

Model provenance

Each saved run records its input snapshot, configuration version, source records, equation/model versions, warnings, and annual computation decomposition. Assumptions are labeled and remain unreviewed.

Secrets and backups

Session, model, SMTP, CAPTCHA, and provider secrets are environment-managed. Admin backup actions and verification are available; operational secrets are not exposed in the UI.

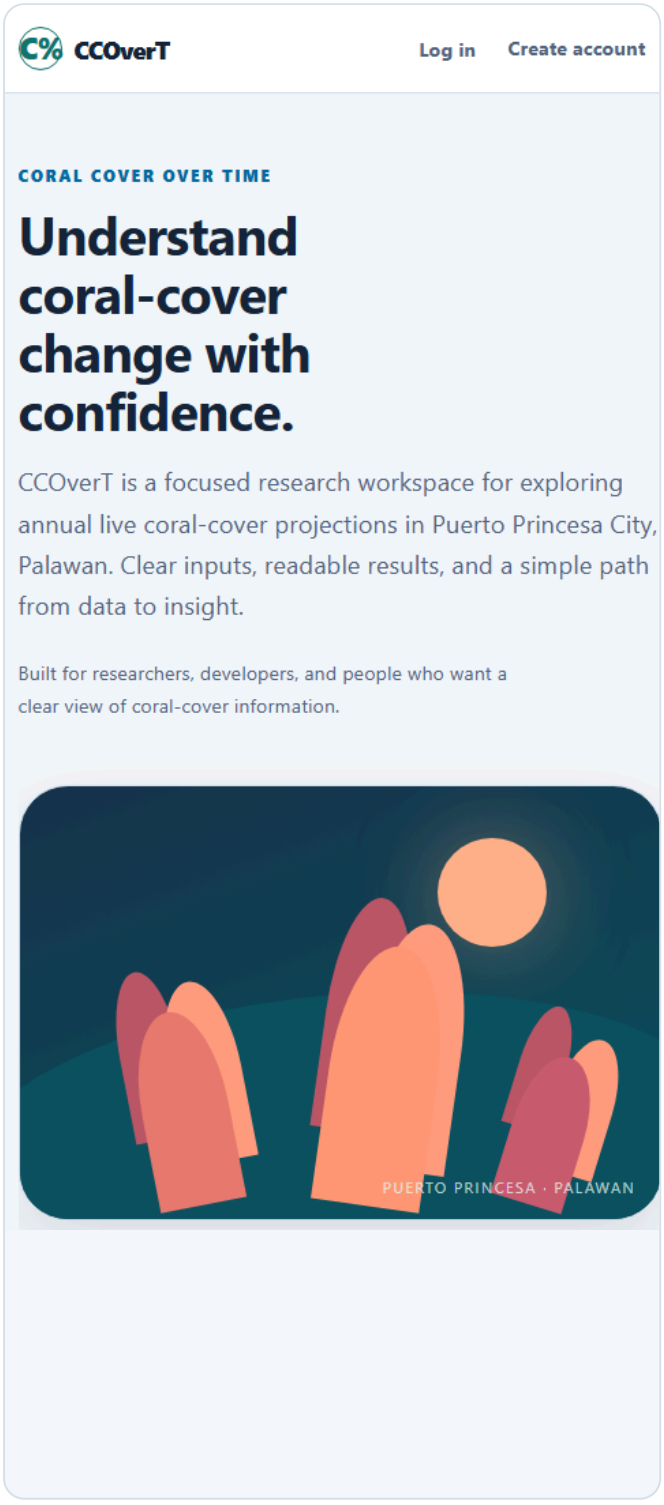
Configuration responsibilities

Use `.env.example` as the variable inventory. Production requires HTTPS origins, a strong session secret, a model service token, persistent database configuration, and non-production demo settings disabled. Keep live credentials in a deployment secret store or ignored environment file.

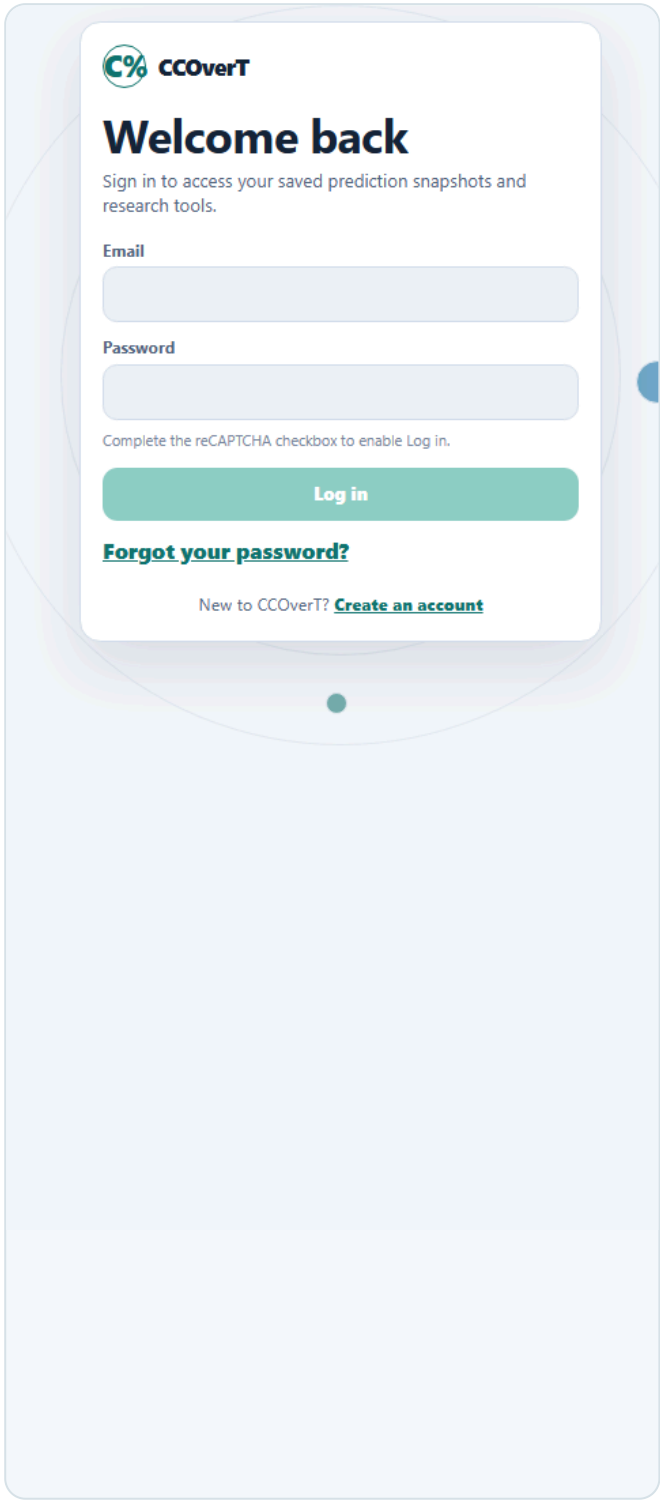
05 · MOBILE SCREENSHOTS

System pages · mobile view

The following pages were captured from the local application at a 480 × 900 CSS-pixel viewport. The landing page is public. The login screen is the unauthenticated entry to protected workspaces; no user data is present in these captures.



Public system landing page · mobile viewport



Sign-in page · mobile viewport